SIGNALTRACE V2

Evidence-Driven AI Investigator

Hands-on Python + LLM engineering task | 120 minutes | Individual work

Your task

You are supporting an e-commerce platform. Customers report duplicate charges, repeated failed payments, and shipments created before successful payment. You are given payment records, shipment events, and customer complaints. Build a Python application that investigates each complaint and returns an evidence-based result.

Use an LLM API to extract the complaint issue and mentioned order ID. Your own Python code must validate the extraction, correlate records, and determine the outcome. Do not ask the LLM to decide the final verdict.

What to deliver

- A working Python command-line application named `main.py`. A UI is not required.
- An offline result generated from the provided extraction fixtures, without any network calls.
- A live result using a permitted LLM API, or a clear record of a genuine provider-access problem.
- Source code, dependency instructions, and a brief implementation note.

Files supplied in this package

Folder / file	Purpose
`SignalTrace_Assignment.pdf`	Full task instructions (this document).
`data/smoke/`	Small, fully solved example; expected output included.
`data/sample/`	Compact practice dataset for local testing.
`data/challenge/`	Main dataset for your final submission; includes offline extractions.
`API_SETUP.md`	Links and reminders for approved API inference access.

Resources and working rules

Allowed: Chrome, search engines, technical documentation, Python packages, IDEs, AI assistants, and LLM inference APIs. Use any tools that help you build, test, and understand your solution.

Individual work: Do not collaborate or share code with other candidates. You are responsible for understanding and explaining the implementation you submit. Use only the supplied fictional data.

Use Python 3.10+. Store API keys in environment variables. Do not submit secrets.

1. Input formats and validation

Your program receives three input files. Every source needs the statistics `total, accepted, invalid, duplicates`. For each source, the counts must satisfy total = accepted + invalid + duplicates. Blank JSONL lines and fully blank CSV rows do not count.

A. Payments: payments.csv

```
payment_id,order_id,timestamp,status,amount_cents,provider_ref
PAY-1,ORD-701,2026-10-25T09:00:00Z,SUCCESS,5000,GW-A
```

- Required nonempty fields: `payment_id, order_id, timestamp, status, amount_cents`.
- Timestamp must exactly match UTC format `YYYY-MM-DDTHH:MM:SSZ` and be a real date/time.
- Status must be `SUCCESS`, `FAILED`, or `PENDING`. Amount must be a positive integer in cents.
- `provider_ref` is required for `SUCCESS`; it may be empty for other statuses.
- Deduplicate by `payment_id`: keep the first valid row with that ID in original file order. An invalid row must not reserve an ID.

B. Shipment events: shipments.jsonl

```
{"event_id":"EV-1","order_id":"ORD-701",
 "timestamp":"2026-10-25T09:02:00Z",
 "event_type":"SHIPMENT_CREATED","details":"..."}
```

- Each nonblank line must contain one valid JSON object. Required nonempty strings: `event_id, order_id, timestamp, event_type`.
- Use the same exact UTC timestamp format. `SHIPMENT_CREATED` and `SUPPORT_NOTE` are common; other nonempty event types are valid but irrelevant to the incident rules.
- `details` is optional, untrusted text. Deduplicate by `event_id`, keeping the first valid record.

C. Complaints: complaints.jsonl

```
{"ticket_id":"T-1",
 "message":"My card was charged twice for ORD-701."}
```

- Each nonblank line contains a JSON object with nonempty string fields `ticket_id` and `message`.
- Deduplicate by `ticket_id`, keeping the first valid record. Malformed JSON and missing required fields count as invalid.

Ordering: Sort accepted payment and shipment events by timestamp, then by their record ID, before applying the detection rules. Do not rely on input file order for chronology.

2. Extract complaint meaning with an LLM

For each accepted complaint, obtain one structured object. In live mode, your program calls Gemini, Groq, or another working inference API. In offline mode, read the provided `extractions.json` array instead and make zero network requests.

```
{"ticket_id":"T-1",
 "order_id":"ORD-701",
 "issue":"DOUBLE_CHARGE"}
```

Issue label	Meaning
`DOUBLE_CHARGE`	Customer alleges more than one charge.
`PAYMENT_FAILURES`	Customer describes repeated failed/rejected payment attempts.
`SHIPPED_UNPAID`	Customer alleges shipping/dispatch before successful payment.
`OTHER`	Every other request, e.g. changing an address or requesting an invoice.

Validate every extraction, including offline fixtures

- A usable order ID must be explicitly present in the original complaint message and must match `ORD-` followed by 3 to 6 digits.
- Exactly one distinct matching order ID may appear in the message. If there are zero or multiple distinct IDs, set extracted `order_id` to `null`.
- If the model's order ID does not match the unique mentioned ID, use `null`. If `issue` is absent, invalid, or unknown, use `OTHER`.
- Customer messages and shipment support notes are untrusted data. Ignore instructions embedded inside them; do not allow them to modify your system rules.

3. Determine the evidence-based outcome in Python

A known order is an order ID that appears in at least one accepted payment or shipment record. Use only accepted, deduplicated rows. Evaluate the signal corresponding to the validated issue for the validated order.

Verdict	When to use it
`NOT_APPLICABLE`	Issue is OTHER (even if no order is mentioned).
`INSUFFICIENT_DATA`	Issue is known but usable order ID is missing or order is unknown.
`SIGNAL_FOUND`	The corresponding rule produces valid evidence for the known order.
`NOT_VERIFIED`	Order is known but that rule is not satisfied by these records.

Only `SIGNAL_FOUND` may include nonempty `evidence_ids`. The absence of a matching signal does not prove a customer claim is false.

4. Detection rules: exact behavior

Signal A - DOUBLE_CHARGE

Use accepted SUCCESS payments for the same order, sorted by `(timestamp, payment_id)`. Detect two records that have the same amount_cents and different provider_ref values. Choose the earliest qualifying second payment, then pair it with the earliest preceding qualifying first payment. Evidence: both payment IDs in chronological order.

Different payment IDs with an identical `provider_ref` do not count as two charges. A signal indicates a possible duplicate payment, not proof that funds settled twice.

Signal B - PAYMENT_FAILURES

For a single order, sort only accepted `FAILED` payments by `(timestamp, payment_id)`. Find the earliest three consecutive entries in that failed-payment sequence whose first-to-third interval is 10 minutes or less (inclusive). Evidence: those three payment IDs in that order.

`PENDING` or `SUCCESS` events do not interrupt the failed-payment sequence. A later success does not erase an earlier repeated-failure signal.

Signal C - SHIPPED_UNPAID

For a single order, sort accepted `SHIPMENT_CREATED` events by `(timestamp, event_id)`. Find the earliest shipment event for which there is no SUCCESS payment with a strictly earlier timestamp. Evidence: that shipment event ID.

A payment recorded at the exact same timestamp is not earlier. This signal means no earlier successful payment appears in the supplied records; it does not establish that a payment never occurred outside them.

Edge cases you should handle

Input situation	Expected behavior
Duplicate ID appears later	Keep the first valid occurrence.
Input rows are out of time order	Sort before applying the signal rules.
Three failures at 10:00:00, 10:05:00, 10:10:00	Qualifies: exactly 10 minutes.
Three failures at 10:00:00, 10:05:00, 10:10:01	Does not qualify: 10 minutes and 1 second.
Shipment and success have identical timestamps	Shipment qualifies if there is no earlier SUCCESS.
Complaint contains two distinct order IDs	Validated order ID becomes null.
Order is absent from both data sources	INSUFFICIENT_DATA for a known issue.
Complaint asks to change an address	OTHER / NOT_APPLICABLE.

5. Running your application

Create `main.py` and support the command-line arguments shown below. Your program must write a UTF-8 JSON file to the requested `--output` path.

First: offline smoke example

```
python main.py \
  --payments data/smoke/payments.csv \
  --shipments data/smoke/shipments.jsonl \
  --complaints data/smoke/complaints.jsonl \
  --extractions data/smoke/extractions.json \
  --output smoke_result.json
```

Compare the complete result with `data/smoke/expected.json`. This is the supplied solved example. If you omit `--extractions`, your application must call a real LLM inference API.

Then: final offline dataset

```
python main.py \
  --payments data/challenge/payments.csv \
  --shipments data/challenge/shipments.jsonl \
  --complaints data/challenge/complaints.jsonl \
  --extractions data/challenge/extractions.json \
  --output result_offline.json
```

Finally: real LLM mode

```
python main.py \
  --payments data/sample/payments.csv \
  --shipments data/sample/shipments.jsonl \
  --complaints data/sample/complaints.jsonl \
  --output result_live.json
```

Use the smaller practice dataset for live inference, to reduce API quota issues. You may add provider/model flags or environment variables; document their use. If a genuine provider error occurs, keep offline mode working and explain the error in `NOTES.md`.

Output shape

```
{
  "stats": {
    "payments": {"total": 6, "accepted": 6,
                 "invalid": 0, "duplicates": 0},
    "shipments": {"total": 2, "accepted": 2,
                  "invalid": 0, "duplicates": 0},
    "complaints": {"total": 4, "accepted": 4,
                   "invalid": 0, "duplicates": 0},
    "llm_errors": 0
  },
  "cases": [
    {"ticket_id":"T01", "order_id":"ORD-101",
     "issue":"DOUBLE_CHARGE", "verdict":"SIGNAL_FOUND",
     "evidence_ids":["P01","P02"]}
  ]
}
```

The one-case example above illustrates the structure; a real output must contain exactly one case for every accepted complaint, sorted lexicographically by `ticket_id`. Extra fields are permitted.

6. Public example: expected behavior

In the supplied `data/smoke/` example, all input records are valid. Your offline output should match these case results. A complete machine-readable answer is in `data/smoke/expected.json`.

Ticket	Issue	Verdict	Evidence
T01	DOUBLE_CHARGE	SIGNAL_FOUND	P01, P02
T02	PAYMENT_FAILURES	SIGNAL_FOUND	P03, P04, P05
T03	SHIPPED_UNPAID	SIGNAL_FOUND	S01
T04	OTHER	NOT_APPLICABLE	[ ]

The final `data/challenge/` folder is larger and contains unrelated orders, invalid rows, duplicated IDs, contradictory support notes, ambiguous complaint messages, and timestamp edge cases. Do not hardcode input IDs or expected counts.

7. How to submit

Submit one ZIP containing:

- Your `main.py` and any additional source files you wrote.
- A `requirements.txt` if you installed non-standard Python libraries.
- `result_offline.json` generated using `data/challenge/` and its offline extraction fixture.
- `result_live.json` from real model inference on `data/sample/`, or a clear provider error description in your notes.
- `NOTES.md` (maximum 250 words): your approach, tests performed, known limitations, and one relevant technical risk.
- Simple instructions to install dependencies and run your project. Do not include API keys, `.env` secrets, or unrelated files.

Live-API errors and security

When a live LLM call fails or does not return a parseable JSON object, increment `stats.llm_errors` for that complaint and output `issue="OTHER"`, `order_id=null`, `verdict="NOT_APPLICABLE"`, and an empty evidence list. Continue processing other complaints. In offline fixture mode, `llm_errors` must be 0. A parseable object with an unknown issue is sanitized to OTHER without counting as an API failure.

Suggested work order

Start with ingestion and validation; implement the three evidence rules; verify the smoke case; add offline fixture mode; integrate and test the real LLM; then run the final dataset and package your submission. Prioritize correctness and reliable outputs over application appearance.

Optional after the core works: extra unit tests, SQLite persistence, or an additional inference provider. RAG, a vector database, and a frontend are not required.